



模板编译

模板是一种泛型编程，本质是一种蓝图，代表了一种“代码生成规则”，在需要实例化某个具体的类型时，会根据该规则生成对应的类，而在实例化之前并不存在该类。

C++ 模板的实例化是**按需编译**、**延迟生成**（**lazy instantiation**）的。

模板的编译不支持声明与定义放在不同的文件中，如果一定要分离，则需要在头文件对实际使用到的模板类型进行显式实例化。

- **模板定义阶段（声明与定义）**
 - 编译器读取模板的声明和定义，存储模板代码，但此时并不生成具体代码。
- **模板实例化阶段**
 - 在编译期，编译器会对确定了具体类型的模板（比如 `std::vector<int>`）进行编译，确定了类型的模板类或模板函数会进行与普通类或普通函数相同的编译方式。
 - 这个过程发生在**编译时**（编译单元内），叫做**即时实例化**（**implicit instantiation**）。
 - 也可以通过显式实例化语句（`template class MyClass<int>;`）提前告诉编译器生成实例。
- **链接阶段**
 - 多个编译单元中相同的模板实例会被链接器合并。

比如模板类 `template<T>`，只有遇到 `template<int>` 时才会将 `template<int>` 进行实例化，此时其他类型的模板并没有被实例化。

对于普通函数和普通类而言，编译期只需要知道函数签名，在链接期再链接具体的实现，该前提下，头文件中的声明足够表示函数签名。

在分离编译的情况下，虽然理论上可以在实例化前根据模板签名推导出对应类型的函数签名，但是模板定义所在的编译单元无法得知其他编译单元需要使用的模板类，即使其他编译单元能够推导出函数签名，也无法通知定义所在的编译单元编译出对应类型的实现。

此外，合法性的检查也是位于定义所在的编译单元中，对于普通函数，在编译初期就会检查其合法性，而模板类的函数会在实例化时检查，此时需要定义所在的编译单元知道要检查的类型，在位于不同编译单元的情况下，这是无法做到的。

本质上，模板不能分离编译，是为了让使用了模板的源文件和模板定义位于同一编译单元，要做到这点，需要在每个使用了模板的源文件中都实现模板的定义（不会构成重定义，因为编译器允许定义等价的模板定义出现在多个编译单元），这样非常麻烦，而在头文件中定义，就相当于在所有包含了该头文件的源文件中定义。

之所以要让使用了模板的源文件和模板定义位于同一编译单元，是因为需要让定义所在的编译单元知道需要实例化的类型，而显式实例化与全特化可以在二者不在同一编译单元的情况下，使定义编译单元知道要实例化的类型，所以显式实例化与全特化可以进行分离编译：

▼ 显式实例化

对指定的类型进行实例化，这样可以在编译开始就实例化，并确定成员函数的函数签名。

```
▼ MyClass.h C++
#pragma once

#include <iostream>

template <typename T>
class MyClass {
public:
    void show();
};

// 告诉编译器：某些具体类型的实例化将在别的地方给出（如 MyClass.cpp）
extern template class MyClass<int>; // 声明：int版本将在其他地方实例化
extern template class MyClass<double>;
```

```
▼ MyClass.cpp C++
#include "MyClass.h"

template <typename T>
void MyClass<T>::show() {
    std::cout << "Type: " << typeid(T).name() << std::endl;
}

// 显式实例化：这里生成具体类型的代码
template class MyClass<int>; // 实例化 MyClass<int>
template class MyClass<double>; // 实例化 MyClass<double>
```

```
▼ main.cpp C++
#include "MyClass.h"

int main() {
    MyClass<int> obj1;
    obj1.show(); // ✅ 使用 MyClass<int>，使用的是 cpp 中显式实例化的定义

    MyClass<double> obj2;
```

```

obj2.show(); // ✅ 同上

// MyClass<char> obj3; // ❌ 会链接失败 ( 没有实例化 char 类型 )
return 0;
}

```

▼ 全特化

确定所有的模板参数，也可以在编译开始时就实例化，还可以进行与通用模板不同的函数定义

```

▼ MyClass.h C++
#pragma once
#include <iostream>

template<typename T>
class MyClass {
public:
    void func() {
        std::cout << "Generic version\n";
    }
};

// 全特化声明 ( 只声明 )
template<>
class MyClass<int> {
public:
    void func(); // 仅声明
};

▼ MyClass.cpp C++
#include "MyClass.hpp"
#include <iostream>

// 全特化定义
void MyClass<int>::func() {
    std::cout << "Specialized version for int\n";
}

▼ main.cpp C++
#include "MyClass.hpp"

int main() {
    MyClass<double> d; // 使用通用版本
    d.func();

    MyClass<int> i;    // 使用特化版本
    i.func();

    return 0;
}

```

模板的实例化发生在编译期，所以在编译期未能进行实例化的模板类型，在运行时也是不能用的。

对于这种情况，可以提前对可能在运行时使用的模板类型进行显式实例化或全特化，主动对其进行实例化。